

JAVA MINI PROJECT

DAILY EXPENSES MANAGER

Project Report

Student Name	Priyanshu Tiwari
Registration Number	24BCY10108
Technology	Java SE
Project Type	Console-Based Application

VIT BHOPAL UNIVERSITY

1. Abstract

An easy-to-use Java console program called the Daily Expenses Manager was created to assist users in keeping track of their daily expenditures. Users can create daily spending reports, set category-specific daily budgets, add expenses with an amount, category, date, and notes, view all recorded expenses, and receive a budget alert when spending surpasses the configured limit. Object-Oriented Programming, ArrayList, HashMap, exception handling, date validation, and file input/output are among the fundamental Java concepts used in this project. There is no need for an external database because expense data is stored locally in an expenses.txt file using a straightforward CSV format.

2. Introduction

Small daily expenses can become challenging to manage when transactions are either forgotten or inconsistently recorded. This project offers a straightforward command-line solution that allows the user to quickly view their current day's spending and keeps all of their expense records in one location. The application emphasizes simplicity and shows how fundamental Java features can be integrated to create a practical real-world tool.

3. Objectives

- Record daily expenses with amount, category, date, and notes.
- Validate user input to reduce invalid expense records.
- Display all stored expenses in a readable format.
- Calculate the total spending for the current day.
- Provide a category-wise breakdown of today's spending.
- Allow users to define daily spending limits for categories.
- Alert the user when category spending exceeds its daily budget.
- Save expense records to a local file and reload them when the application starts.
- Demonstrate core Java, OOP, collections, exception handling, and file I/O concepts.

4. Technology Stack

Component	Technology	Purpose
Programming Language	Java SE	Application development
Collections	ArrayList, HashMap	Store expenses and budgets
Date API	java.time	Date validation and current date
File I/O	java.io	Persistent local storage
Input	Scanner	Interactive console input
Dependencies	None	Uses only core Java

5. Functional Requirements

Function	Description
Add Expense	Accept amount, category, date, and notes; validate amount/date; add the expense.
View Expenses	Display every recorded expense in a formatted list.
Daily Report	Calculate today's total and category-wise totals.
Set Budget	Store a daily budget limit against a category.
Budget Alert	Compare category spending for a date with its configured limit.
Save & Exit	Write all expenses to expenses.txt before closing.

Load Data	Read existing expense records from expenses.txt at startup.
-----------	---

6. System Design / Working

The application follows a menu-driven flow:

1. Start the application and load existing records from expenses.txt.
2. Display the main menu with five operations.
3. The user selects an operation using options 1–5.
4. For Add Expense, the program validates the amount and date before creating an Expense object.
5. The expense is added to the ArrayList and the budget is checked for the same category and date.
6. For Daily Report, expenses matching today's date are aggregated by total and category.
7. For Set Budget, the category and daily limit are stored in a HashMap.
8. On Save & Exit, each expense is converted to CSV and written to expenses.txt.

7. Data Structures and OOP Design

The program defines an inner Expense class containing amount, category, date, and notes. Its constructor initializes these fields, getter methods provide access, toCSV() supports file persistence, and toString() provides formatted console output.

Structure	Use in Project	Benefit
Expense class	Represents one expense record	Encapsulation and clear data model
ArrayList<Expense>	Stores all expense records	Dynamic collection and easy iteration
HashMap<String, Double>	Stores category budget limits	Fast category-to-budget mapping
String	Category, date, notes	Simple text representation

8. Input Validation and Exception Handling

The Add Expense function repeatedly asks for the amount until a valid non-negative number is entered. NumberFormatException is handled when numeric input cannot be parsed. Dates are validated using LocalDate.parse(), and DateTimeParseException is handled when the entered date does not follow YYYY-MM-DD. Budget input uses similar validation for non-negative numeric values.

9. File Handling and Data Persistence

Expense data is stored in the local file expenses.txt. During loading, BufferedReader and FileReader read each line, which is split into four CSV fields. During saving, PrintWriter and FileWriter write each expense using its toCSV() representation. The implementation also handles IOException and skips malformed numeric records with a warning.

10. Key Modules / Methods

Method	Responsibility
main()	Loads data, displays the menu, and routes the selected operation.
addExpense()	Collects and validates expense details, creates an Expense object, and checks the budget.
viewExpenses()	Prints all recorded expenses.
showDailyReport()	Calculates today's total and category-wise spending.

setBudget()	Stores a daily budget for a category.
checkBudget()	Calculates spending for the category/date and prints an alert if the limit is exceeded.
loadExpenses()	Loads saved expense records from expenses.txt.
saveExpenses()	Writes all current expense records to expenses.txt.

11. Main Menu

1. Add Expense
2. View Expenses
3. Daily Report
4. Set Budget
5. Save & Exit

12. Example Usage

Example scenario: a user adds a food expense of \$15.50 for the current date with a note such as Lunch. If the Food category has a daily budget and the new spending causes the total to exceed that limit, the program prints a clear budget-exceeded alert showing the category, budget limit, and amount spent today.

13. Advantages

- Simple and beginner-friendly console interface.
- No external libraries or database setup is required.
- Uses persistent local storage so records survive program restarts.
- Provides category-wise daily reporting.
- Includes validation for common invalid inputs.
- Demonstrates multiple core Java concepts in a single practical application.

14. Limitations

- The application is console-based and does not provide a graphical interface.
- Data is stored in a simple CSV-style text file rather than a database.
- The current report focuses on the current day only.
- There is no authentication or multi-user support.
- Budget information is held in memory and is not persisted to the expenses file.

15. Future Enhancements

- Add a graphical user interface using JavaFX or Swing.
- Introduce monthly and weekly reports with charts.
- Persist budgets along with expenses.
- Use a relational database for larger datasets.
- Add search, edit, and delete functionality for expense records.
- Add export options such as PDF or Excel.
- Provide authentication and separate user profiles.

16. Conclusion

The Daily Expenses Manager demonstrates how a practical expense-tracking utility can be implemented using core Java. It combines an Expense data model with collections, input validation, exception handling, date processing, budget checking, and file persistence. The project provides the essential functionality required for recording and reviewing daily expenses while remaining simple enough to understand and extend.

17. References / Project Files

The report is based on the submitted ExpensesManager.java source code and its project README. The README identifies the project as a core-Java console application for daily expense tracking, budgets, and reports.

Submitted source files:

- ExpensesManager.java — main Java implementation.
- README.md — project overview, features, technology stack, and setup information.

Submitted by
Priyanshu
Tiwari
24BCY10108